Database Encryption Documentation

Dental Clinic Management Application (clinc)

Project	Dental Clinic (clinc) — Flutter Desktop App
Encryption Engine	SQLCipher (AES-256 page-level encryption)
Backup Encryption	AES-256-CBC with HMAC-SHA256 verification
Key Derivation	SHA-256(password) → 256-bit key
Key Storage	RAM only — never persisted to disk
Platforms	macOS, Windows, Linux, Android, iOS
Date	February 2026

Table of Contents

1. Overview

2. Architecture

3. Password Handling & Key Derivation

4. SQLCipher Database Encryption

5. Encrypted Backup System

6. Key Lifecycle Management

7. Cross-Platform Support

8. Security Analysis

9. Summary

1. Overview

The Dental Clinic Management Application uses a **multi-layered encryption strategy** to protect sensitive patient medical data. The encryption is designed around two core principles:

- **1. Zero plaintext on disk:** The database file is always encrypted. There is never a moment where patient data sits unencrypted on the file system — not during normal operation, not during crashes, and not in backup files.
- **2. Key never persisted:** The encryption key exists only in RAM while the app is running. It is derived from the user's password on each login and securely wiped (overwritten with zeros) when the app closes.

This means that even if an attacker gains full access to the device's file system, they will find only encrypted data and a double-hash of the password — neither of which can be used to reconstruct the encryption key or the original password.

Technologies Used

Component	Technology	Purpose
Database encryption	SQLCipher	Page-level AES-256 encryption of SQLite DB
Key derivation	SHA-256 (crypto package)	Password → 256-bit key
Password verification	SHA-256 double hash	hash(hash(pwd)) stored in SharedPrefs
Backup encryption	AES-256-CBC (encrypt package)	Encrypt exported backup files
Backup integrity	HMAC-SHA256	Verify password before decryption attempt
Cross-platform libs	sqlcipher_flutter_libs	SQLCipher binaries for all platforms

2. Architecture

The diagram below shows the high-level architecture of the encryption system. When the user enters their password, three things happen simultaneously:

1. **Verification:** The password is double-hashed (SHA-256 applied twice) and compared against the stored double-hash to confirm the user knows the correct password.
2. **Key derivation:** The password is hashed once (SHA-256) to produce a 256-bit encryption key that is stored in RAM only.
3. **Database unlock:** The key is converted to a hex string and passed to SQLCipher via `PRAGMA key`, which unlocks every page of the database.

Figure 1: High-Level Encryption Architecture

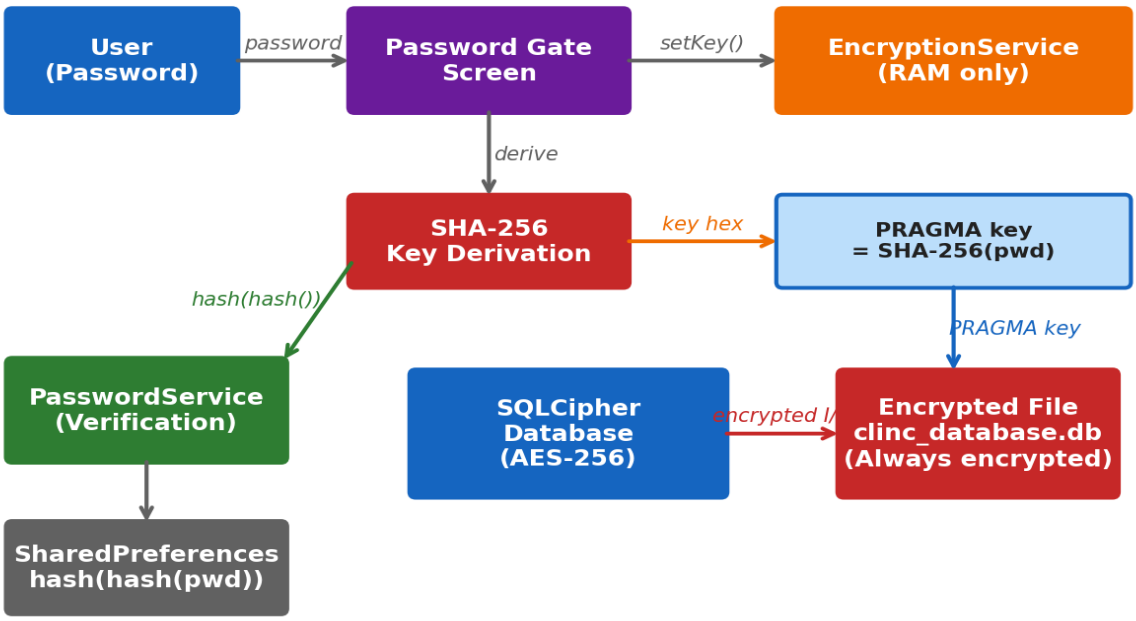

Figure 1: High-level encryption architecture showing the flow from user password to encrypted database.

3. Password Handling & Key Derivation

3.1 What Happens When You Enter Your Password

When the user enters their password on the login screen, the application processes it through two parallel paths. This is a critical design choice — the **verification path** and the **encryption path** use different hash depths, so the stored verification hash cannot be used to derive the encryption key.

Figure 2: Password Handling & Key Derivation

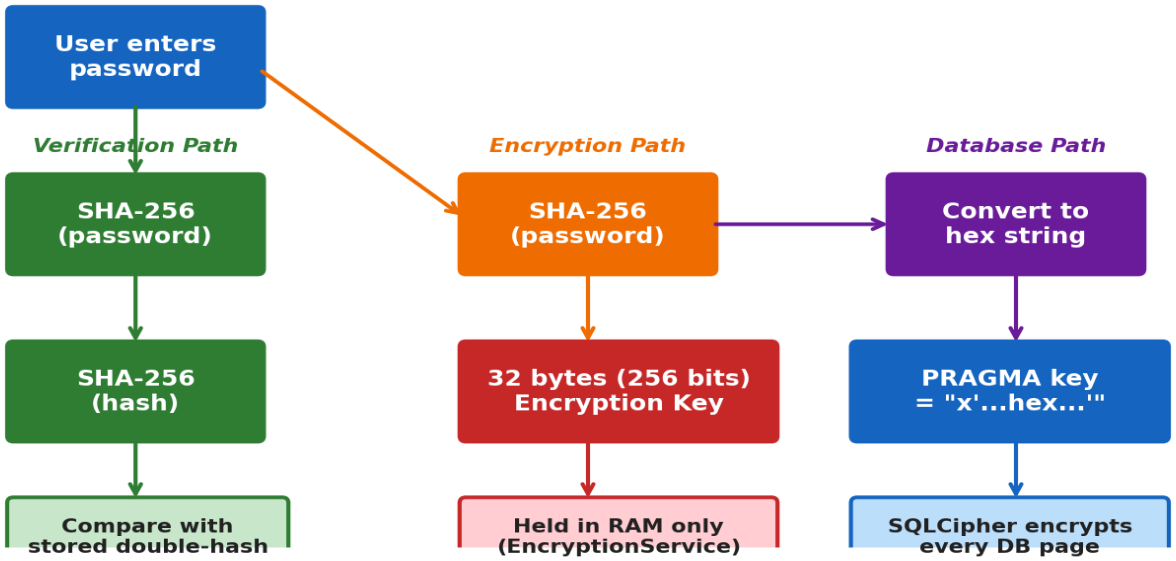

Figure 2: The password is processed through two independent paths — one for verification, one for encryption.

3.2 Why Double-Hash for Verification?

The encryption key is `SHA-256(password)`, which is a single hash. If we stored this same single hash on disk for verification, an attacker who reads the SharedPreferences file would have the encryption key directly! Instead, we store `SHA-256(SHA-256(password))` — a double hash. SHA-256 is a one-way function, so knowing hash(hash(password)) does not help an attacker compute hash(password) (the encryption key).

3.3 Key Derivation Detail

Step	Operation	Result	Stored?
1	User enters password "MyP@ss"	"MyP@ss"	Never
2	SHA-256("MyP@ss")	32 bytes (encryption key)	RAM only
3	SHA-256(result of step 2)	32 bytes (double hash)	SharedPreferences

4. SQLCipher Database Encryption

4.1 How SQLCipher Works

SQLCipher is a fork of SQLite that adds transparent, page-level encryption using AES-256. When enabled with a key, SQLCipher encrypts **every page** of the database file individually. This means:

- The database file on disk is **always encrypted** — there is no plaintext phase.
- Encryption/decryption happens transparently when reading/writing pages.
- The application code uses normal SQL queries — SQLCipher handles all crypto.
- No performance impact is visible because encryption operates at the page level (4KB chunks).

Figure 3: SQLCipher Page-Level Encryption

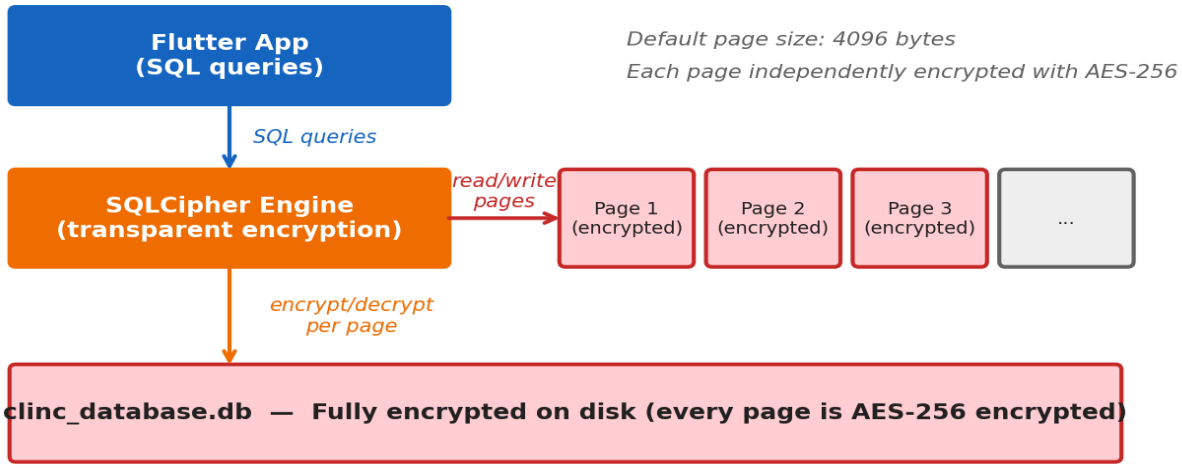

Figure 3: SQLCipher encrypts each database page independently with AES-256.

4.2 PRAGMA key Configuration

The encryption key is set on every database connection via the `onConfigure` callback, which runs before any other SQL statement:

```
Future _onConfigure(Database db) async {
  final keyHex = EncryptionService.instance.keyHex;
  if (keyHex != null) {
    await db.execute("PRAGMA key = \"x'$keyHex'\"");
  }
  await db.execute('PRAGMA foreign_keys = ON');
}
```

The `x'...'` prefix tells SQLCipher that the key is provided as raw hex bytes (64 hex characters = 32 bytes = 256 bits), rather than as a passphrase that would need additional key derivation.

4.3 Password Change (PRAGMA rekey)

When the user changes their password, the database is re-encrypted in place using `PRAGMA rekey`. This re-encrypts every page of the database with the new key without requiring an export/import cycle:

```
Future<void> rekeyDatabase() async {
  final keyHex = EncryptionService.instance.keyHex;
  final db = await database;
  await db.execute("PRAGMA rekey = \"x'$keyHex'\"");
```

```
}
```

5. Encrypted Backup System

5.1 Export: Creating an Encrypted Backup

When the user exports data, the application creates a `.clinc` file (a custom encrypted format). The process has four stages:

1. **Query:** All table data (Patients, Appointments, Treatments, Invoices, Payments, Expenses) is queried from the SQLCipher database.
2. **Serialize:** Data is serialized to JSON. X-ray images are collected alongside it.
3. **Compress:** JSON and images are compressed into a ZIP archive (in a temp directory).
4. **Encrypt:** The entire ZIP is encrypted with AES-256-CBC using the current password's derived key, with a random 16-byte IV and an HMAC-SHA256 verification tag.

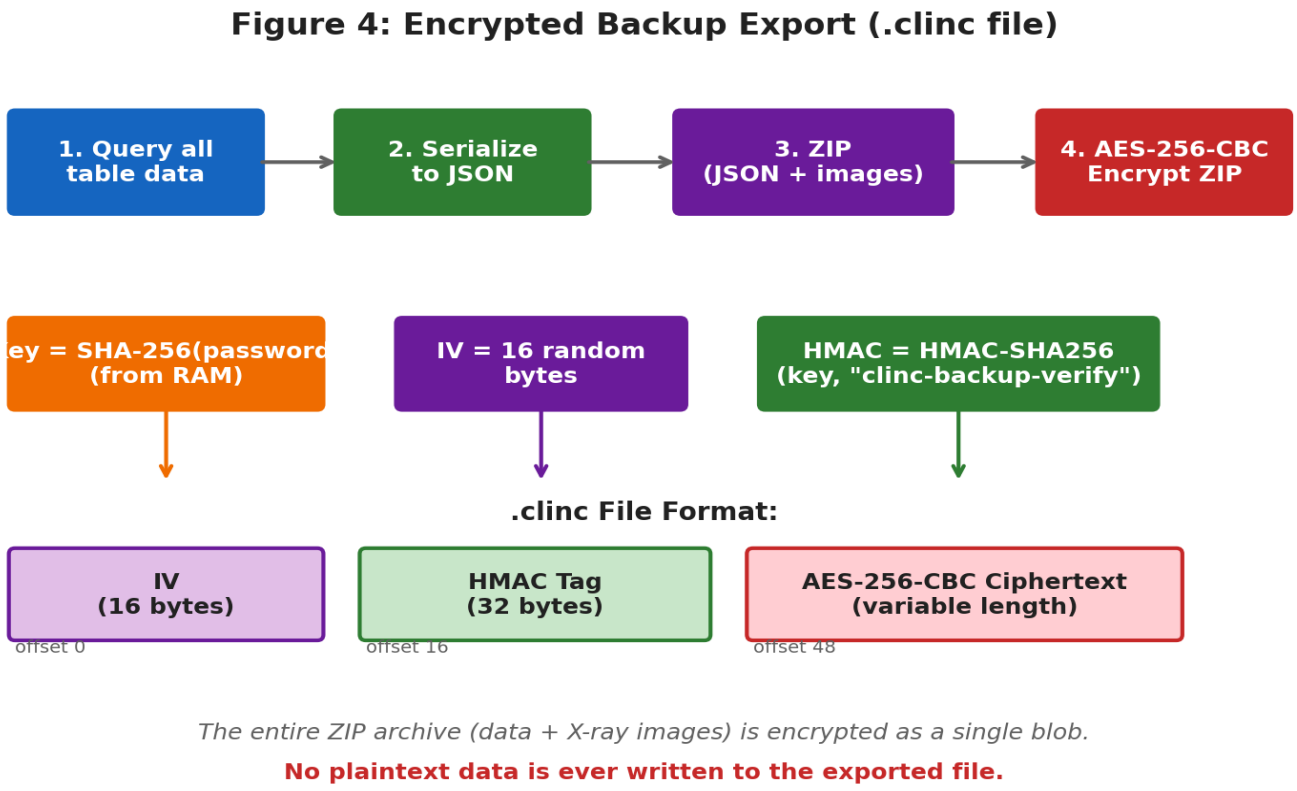

Figure 4: The export process — data → JSON → ZIP → AES-256-CBC encrypted .clinc file.

5.2 The .clinc File Format

The exported file has a simple binary format:

Offset	Size	Content	Purpose
0	16 bytes	IV (Initialization Vector)	Random nonce for AES-CBC (unique per export)
16	32 bytes	HMAC-SHA256 tag	Quick password verification without decrypting
48	Variable	AES-256-CBC ciphertext	The encrypted ZIP containing all backup data

5.3 HMAC Verification Tag

The HMAC tag allows the app to verify whether the entered password is correct **before** attempting to decrypt the potentially large ciphertext. The tag is computed as:

```
HMAC-SHA256(key=SHA-256(password), message="clinc-backup-verify")
```

During import, the app derives the key from the entered password, recomputes the HMAC, and compares it with the stored tag using **constant-time comparison** (to prevent timing attacks). If they don't match, the wrong password error is shown immediately.

5.4 Import: Restoring from an Encrypted Backup

Importing a `.clinc` backup is the reverse process. The app first asks the user for the password that was active when the backup was created. This is critical: if you change your password and then try to import an old backup, you must enter the **old** password.

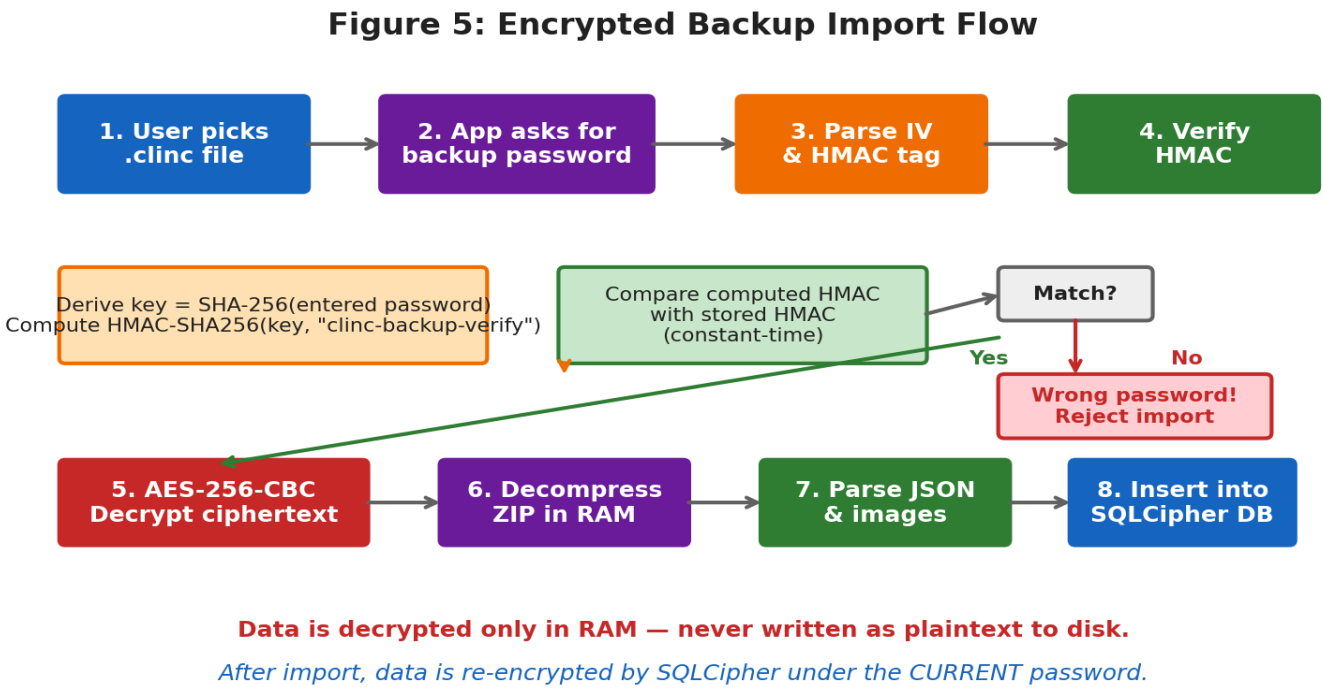

Figure 5: The import flow — .clinc file → password verification → decryption in RAM → insert into current SQLCipher DB.

After the data is decrypted in memory, it is inserted into the current SQLCipher database, which is encrypted with the **current** password. The plaintext data exists only in RAM for the duration of the import operation and is never written to disk.

5.5 Legacy Backup Compatibility

The app also supports importing plain `.zip` backup files from before the encryption feature was added. These are imported directly without any password prompt. The file picker accepts both `.clinc` and `.zip` extensions.

6. Key Lifecycle Management

The encryption key follows a strict lifecycle designed to minimize the window during which it exists in memory:

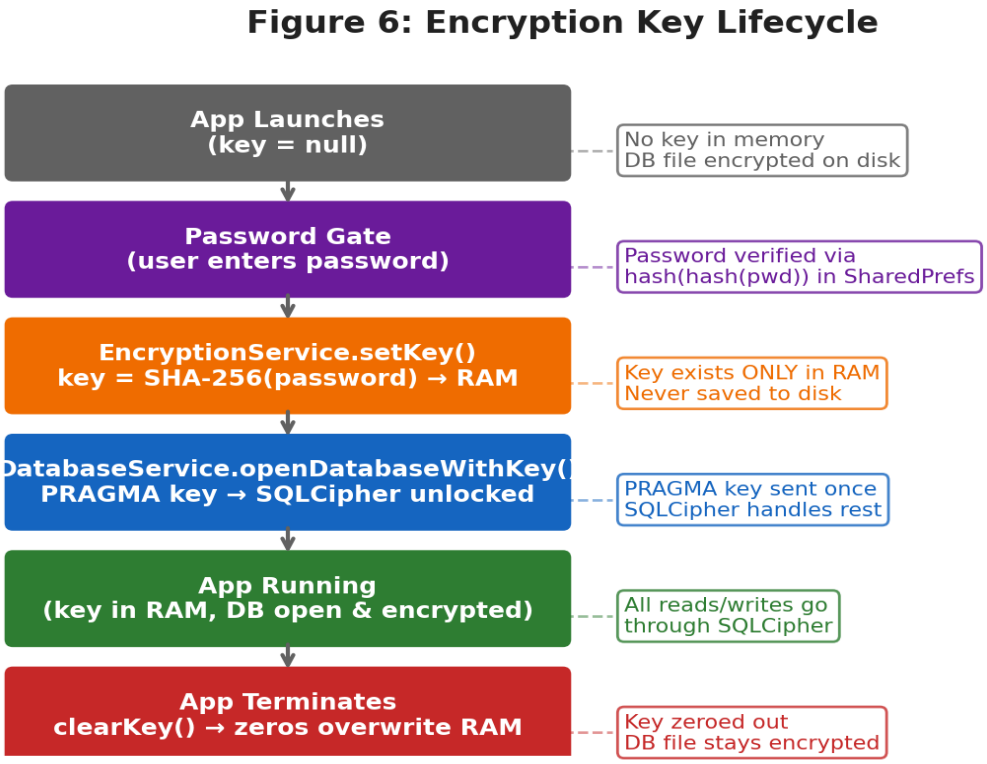

Figure 6: The encryption key lifecycle from app launch to termination.

Key Lifecycle Events

Event	Action	Key State
App launches	Nothing — wait for password	null (no key)
User enters password	EncryptionService.setKey(pwd)	SHA-256(pwd) in RAM
DB opens	PRAGMA key = hex(key)	Key sent to SQLCipher
App running normally	All SQL goes through SQLCipher	Key in RAM
Password changed	PRAGMA rekey with new key	New key replaces old
App detached/terminated	clearKey() → zeros overwrite	null (wiped)
App crash	OS reclaims memory	RAM freed by OS

The `clearKey()` method doesn't just set the reference to null — it **overwrites every byte with 0x00** before nulling the reference. This prevents the key from lingering in memory even briefly after the app decides to clear it.

7. Cross-Platform Support

SQLCipher libraries are loaded differently on each platform. The app handles this in `main()` before any database access:

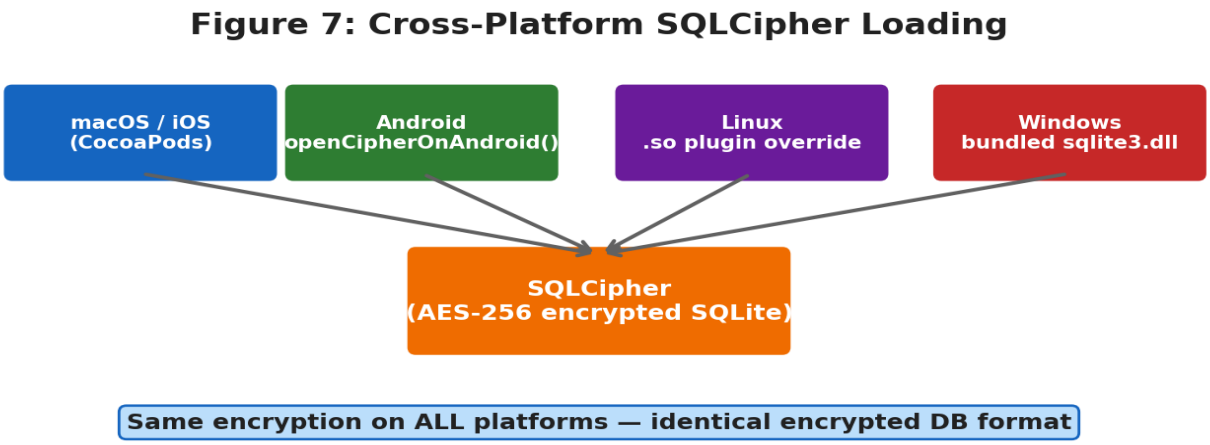

Figure 7: Platform-specific SQLCipher library loading strategies.

Platform	Library Loading	Notes
macOS	Automatic (CocoaPods)	SQLCipher linked as a native framework via Podfile
iOS	Automatic (CocoaPods)	Same as macOS — CocoaPods resolves the dependency
Android	openCipherOnAndroid()	Workaround for old Android versions; loads .so from APK
Linux	DynamicLibrary.open(path)	Absolute path to libsqlcipher_flutter_libs_plugin.so
Windows	Automatic (bundled DLL)	sqlite3.dll (SQLCipher build) bundled in app directory

Despite the different loading mechanisms, the encryption is **identical** across all platforms. A `.clinc` backup created on macOS can be imported on Windows or Linux and vice versa.

8. Security Analysis

8.1 What Is Stored Where

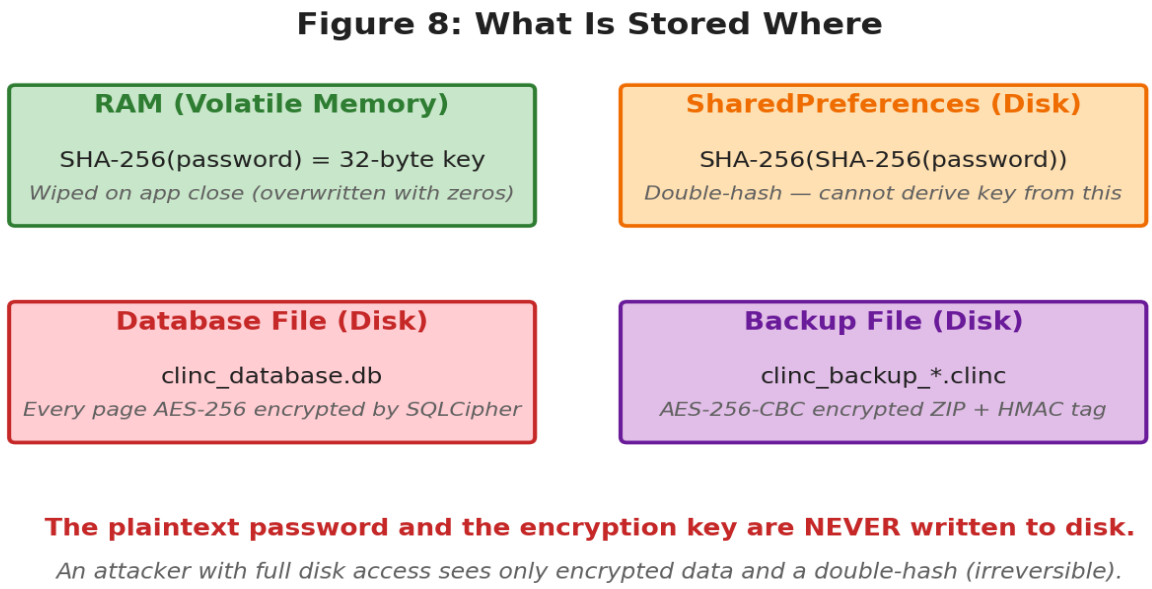

Figure 8: Summary of what data is stored in RAM vs. on disk.

8.2 Threat Scenarios

Threat	Protection	Result
Attacker reads DB file	SQLCipher AES-256 page encryption	Sees only encrypted bytes
Attacker reads SharedPrefs	Only double-hash stored	Cannot derive encryption key
Attacker reads .clinc backup	AES-256-CBC + HMAC	Cannot decrypt without password
Attacker tries wrong password on backup import	HMAC verification + constant-time comparison	Rejected immediately; no timing leak
App crashes mid-operation	SQLCipher keeps file encrypted	DB file never plaintext on disk
RAM dump while app open	Key is 32 bytes in one location	Requires root access + RAM forensics
App terminates normally	clearKey() zeros memory	Key wiped from RAM

8.3 Design Decisions Explained

Q: Why SHA-256 for key derivation instead of PBKDF2/Argon2?

A: SQLCipher itself applies its own key derivation (PBKDF2 with 256,000 iterations by default) on top of whatever key we provide. So our SHA-256 output goes through PBKDF2 again inside SQLCipher before being used as the actual page encryption key. The effective security is SHA-256 + PBKDF2(256K rounds).

Q: Why HMAC instead of just trying to decrypt?

A: Without HMAC, a wrong password would cause AES-CBC to produce garbage output, which might crash the ZIP decompressor with an unhelpful error. HMAC gives clear, immediate feedback.

Q: Why constant-time HMAC comparison?

A: Regular byte comparison can leak information about which byte differed first. An attacker measuring response time could narrow down the HMAC byte by byte. Constant-time comparison always takes the same time regardless of which bytes match.

Q: Why AES-256-CBC for backups instead of GCM?

A: The encrypt package provides a well-tested CBC implementation with PKCS7 padding. Combined with our separate HMAC-SHA256 tag, this provides both confidentiality and integrity — equivalent to authenticated encryption. GCM would work too but isn't necessary.

9. Summary

The Dental Clinic application implements a comprehensive, multi-layered encryption system that ensures patient data is protected at all times:

- ✓ **Database:** SQLCipher provides transparent AES-256 page-level encryption. The database file is never plaintext on disk.
- ✓ **Key management:** The encryption key (SHA-256 of the password) lives only in RAM and is securely wiped on app termination.
- ✓ **Password storage:** Only a double-hash is persisted — an attacker cannot derive the encryption key from it.
- ✓ **Backup files:** Exported as AES-256-CBC encrypted .clinc files with HMAC-SHA256 verification. Importing requires the original backup password.
- ✓ **Password changes:** PRAGMA rekey re-encrypts the entire database in place with the new key. Old backups remain encrypted with the old password.
- ✓ **Cross-platform:** Identical encryption on macOS, Windows, Linux, Android, and iOS.
- ✓ **Legacy support:** Old unencrypted .zip backups can still be imported for backward compatibility.

End of Document